BLG223E
Homework-1

Assistant Professor Dr. Yusuf Huseyin Sahin
sahinyu@itu.edu.tr

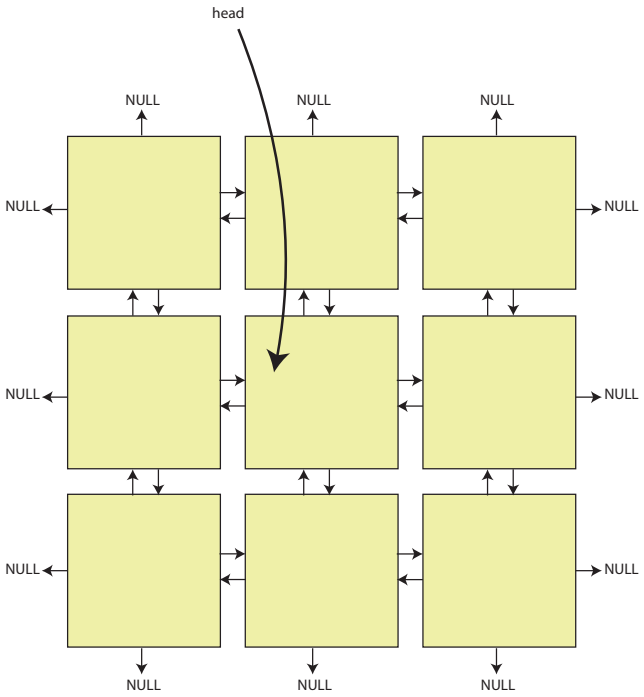


Figure 1: The Cell Grid data structure

In this homework, you will create a new data structure which represents a two-dimensional grid where each element (called a Node) is linked to its four immediate neighbors: up, down, left, and right. This creates a four-directionally connected network of nodes that can be traversed horizontally and vertically. The main distinction of this design is that instead of referencing a corner node, the structure maintains a pointer called "head", which always points to the center node of the grid.

Each node contains a data field (e.g., an integer value) and four directional pointers:

```c
typedef struct Node {
    int value;
    struct Node *up, *down, *left, *right;
} Node;
```

Using the Node data structure, the CellGrid could be defined as:

```c
typedef struct {
    Node *head;     // Always points to the center node
    size_t rows, cols;
} CellGrid;
```

• Implement **CellGrid *gridcreate(int rows, int cols, int initvalue)**. Creates a CellGrid with positive dimensions and fills every cell with initvalue. Set all links and select head using the center rule below. Return NULL for nonpositive dimensions or allocation failure; release any partially allocated grid before returning NULL.

• Implement **void gridupdate(CellGrid *g, int col, int row, int newvalue)**. Updates the value at the given relative coordinate. Use getnode to locate the node. Invalid coordinates or a NULL grid must leave the grid unchanged.

• Implement **void deleterow(CellGrid *g, int rowindex)**. Deletes the row at the given relative row coordinate, reconnects surviving neighbors, frees removed nodes, updates rows, and reselects head.

• Implement **int sumgrid(CellGrid *g)**. Returns the sum of all node values, counting each node exactly once. Return 0 if g is NULL. Assume the mathematical sum fits in an int.

• Implement **Node *getnode(CellGrid *g, int col, int row)**. Returns the node at the given relative coordinate. Return NULL if g is NULL or the coordinate is outside the grid.

• Implement **void deletecol(CellGrid *g, int colindex)**. Deletes the column at the given relative column coordinate, reconnects surviving neighbors, frees removed nodes, updates cols, and reselects head.

• Implement **void gridfree(CellGrid *g)**. Frees every node and then the CellGrid structure. gridfree(NULL) must do nothing. The caller must not access the grid after it has been freed.

Coordinate System and Center Selection

Number physical rows from top to bottom and columns from left to right, starting at zero. The head must point to physical row rows / 2 and physical column cols / 2, using integer division. For an even dimension, choose the lower or right-hand central position.

All public coordinates are relative to the current head: (col, row) = (0, 0). Moving right increases col; moving down increases row. Left and upward coordinates are negative. Where both coordinates are required, the column argument comes first.

For a 4 x 6 grid, head is at physical row 2, column 3. Valid relative row coordinates are -2 through 1, and valid relative column coordinates are -3 through 2. In general, the ranges are -(rows / 2) through rows - 1 - (rows / 2), and -(cols / 2) through cols - 1 - (cols / 2). Evaluate these ranges as signed integers; avoid unsigned subtraction.

Deletion Rules

Interpret rowindex and colindex relative to head before the operation. Preserve the values and physical order of all surviving nodes. After a successful deletion, update the corresponding dimension and reselect head using the center rule above. Future coordinates are relative to this new head.

If g is NULL or the deletion index is invalid, leave the grid unchanged. Do not delete the last remaining row or the last remaining column. Such requests must also leave the grid unchanged.

Implementation Requirements

Use the prescribed Node and CellGrid structures and implement all seven functions with the specified names, return types, and parameter order. Additional helper functions are allowed.

Allocate each node separately. Do not store node addresses in a persistent array, matrix, or external lookup table. Temporary helper storage during construction is allowed, but must be released before gridcreate returns.

All neighbor links must be reciprocal: if a->right is b, then b->left must be a; the same applies to up and down. Pointers facing outside the grid must be NULL. Avoid memory leaks, invalid accesses, and double frees.

Suggested Checks

Test odd and even dimensions, boundary coordinates, invalid operations, center row and column deletion, and single-row or single-column grids. A 3 x 3 grid initialized to 1 has sum 9; updating (1, -1) to 5 gives sum 13. Independently, deleting row 0 from a 3 x 3 grid initialized to 1 produces a 2 x 3 grid with sum 6 and head at physical row 1, column 1.